

Deliverable 3 ? Final Implementation & Security Testing Report

Course: CY321 ? Secure Software Design with Secure Coding Implementation

Deliverable: D3 ? Final Hardened Implementation & Security Analysis

Project: AUI Secure ? DevSecOps CI/CD Pipeline & Secure Coding Demonstration Platform

Date: 2026-05-07

1. What Deliverable 3 Required

Deliverable 3 is the final submission. It must show:

- A hardened version of the application with all security issues resolved
- A complete security code review (manual analysis of every module)
- Static analysis results (SAST ? Bandit)
- Dynamic analysis results (DAST ? OWASP ZAP + manual penetration testing)
- A full test report (pytest)
- A changelog of all security fixes applied between D2 and D3

The D3 documents are in Deliverable-3/:

- Final_Report.md ? Overall project summary
 - Secure_Code_Review.md ? Per-module security review
 - Security_Testing_Analysis.md ? Testing results across all tools
 - Security_Fixes_Changelog.md ? Before/after diffs for each fix
-

2. What Changed Between D2 and D3

The D3 phase began with a full security review of the D2 codebase. Six issues were found and fixed. These are the most important part of D3 ? they show that the review process actually works.

2.1 Fix F1 ? HIGH ? IDOR Attack Endpoint Not Gated (profiles.py)

The problem in D2:

The profile attack endpoint (/users/attack/<id>) was reachable in production ? it had no SECURE_MODE check. Anyone who knew the URL could:

- Access any other user's profile just by changing the ID number
- Inject SQL like ' OR '1'='1 to dump the entire users table
- Get admin's password hash (even though it is script-hashed, this is a serious leak)

D2 code (vulnerable):

```
@bp.get("/attack/<path:user_input>")
@requires_auth
def get_user_profile_insecure(user_input: str):
    # No SECURE_MODE check ? always reachable!
    sql = f"SELECT ... FROM users WHERE id = '{user_input}'"
    users = db.execute(sql).fetchall()
```

D3 code (fixed):

```
@bp.get("/attack/<path:user_input>")
```

```
@requires_auth
def get_user_profile_insecure(user_input: str):
    if current_app.config.get("SECURE_MODE", True):
        return jsonify({"blocked": True, "reason": "..."}), 403
    # Only reachable when SECURE_MODE=false (demo mode)
    sql = f"SELECT ... FROM users WHERE id = '{user_input}'"
```

OWASP mapping: A01 Broken Access Control + A03 Injection

Verified: `curl /users/attack/1` with `SECURE_MODE=true` returns 403. With `SECURE_MODE=false` returns Alice's data (as intended for demo). All 62 tests still pass.

2.2 Fix F2 ? MEDIUM ? `time.sleep()` DoS Amplifier (auth.py)

The problem in D2:

On every failed login, the code called `time.sleep(min(0.5 * attempts, 5.0))`. This blocked the worker thread for up to 5 seconds per request. An attacker with a botnet of N IP addresses could each tie up a Gunicorn worker for 5 seconds ? effectively a DoS attack using the app's own "security" feature against it.

D2 code (vulnerable):

```
if not row or not check_password_hash(row["password_hash"], password):
    _record_failure(db, username)
    time.sleep(min(0.5 * attempts, 5.0)) # BLOCKS worker thread
    return jsonify({"error": "invalid credentials"}), 401
```

D3 code (fixed):

```
if not row or not check_password_hash(row["password_hash"], password):
    if not row:
        # Dummy hash check ? equalises response time for unknown vs known users
        check_password_hash("scrypt:32768:8:1$dummysalt$" + "0" * 128, password)
    _record_failure(db, username)
    return jsonify({"error": "invalid credentials"}), 401
```

The sleep was removed entirely. The timing equalisation is achieved by the dummy hash check (~10ms for both known and unknown users) instead of an artificial sleep.

OWASP mapping: A07 Identification and Authentication Failures (DoS amplifier)

Verified: Response time for known-user-wrong-password and unknown-user are now both ~10ms. No sleep. All lockout tests still pass.

2.3 Fix F3 ? MEDIUM ? `SECURE_MODE` Toggle Open to Any User (lab.py)

The problem in D2:

The endpoint to toggle `SECURE_MODE` was decorated with `@requires_auth` only ? meaning any logged-in user could flip it. Combined with F1, the attack chain was:

- Register as a regular user
- POST `/lab/toggle-secure-mode` (any authenticated user could do this)
- `SECURE_MODE` is now false
- GET `/users/attack/` OR `'1'='1'` ? dumps entire users table including admin

D2 code (vulnerable):

```
@bp.post("/toggle-secure-mode")
```

```
@requires_auth # ANY logged-in user can toggle
def toggle_secure_mode():
    new_state = not current_app.config.get("SECURE_MODE", True)
    current_app.config["SECURE_MODE"] = new_state
```

D3 code (fixed):

```
@bp.post("/toggle-secure-mode")
@requires_role("admin") # ADMIN ONLY
def toggle_secure_mode():
    new_state = not current_app.config.get("SECURE_MODE", True)
    current_app.config["SECURE_MODE"] = new_state
    log_action("SECURE_MODE_TOGGLED", details={"from": ..., "to": new_state})
```

The try/except: print(e) wrapper was also removed ? it was leaking stack traces to stdout in some deployment configurations.

OWASP mapping: A01 Broken Access Control

Verified: Login as alice (role=user) ? POST /lab/toggle-secure-mode ? 403. Login as admin ? toggle succeeds.

2.4 Fix F4 ? LOW ? Import Placed Mid-File (items.py)

The problem in D2:

A from .validators import ... statement appeared after a function definition. This is a Python anti-pattern ? all imports should be at the top of the file.

D2 code:

```
from .audit import log_action
from .db import tx
from .rbac import requires_auth

def _uid():
    return session.get("user_id") or getattr(g, "api_user", {}).get("id")

from .validators import ValidationError, validate_item_body, ... # MID-FILE!
```

D3 code:

```
from .audit import log_action
from .db import tx
from .rbac import requires_auth
from .validators import ValidationError, validate_item_body, ... # AT TOP

def _uid():
    return session.get("user_id") or getattr(g, "api_user", {}).get("id")
```

Verified: All CRUD and search tests still pass.

2.5 Fix F5 ? LOW ? No Admin Demo Account (db.py)

The problem in D2:

The seed data only created 4 regular users (alice, bob, charlie, diana). There was no admin account. This

meant:

- Admin dashboard could not be demonstrated end-to-end
- The `@requires_role("admin")` paths could only be tested for the deny path (403), not the allow path (200)
- The `SECURE_MODE` toggle (after F3 fix) had no way to be used in demos

D2 seed data:

```
demo_users = [  
    (1, "alice", ..., "user", ...),  
    (2, "bob", ..., "user", ...),  
    (3, "charlie", ..., "user", ...),  
    (4, "diana", ..., "user", ...),  
    # No admin!  
]
```

D3 seed data:

```
demo_users = [  
    (1, "alice", ..., "user", ...),  
    (2, "bob", ..., "user", ...),  
    (3, "charlie", ..., "user", ...),  
    (4, "diana", ..., "user", ...),  
    (5, "admin", generate_password_hash("Admin@1234"), "admin", ...),  
]
```

Verified: Login as admin / Admin@1234 ? GET /dashboard ? 200 with admin UI. Login as alice ? 403.

2.6 Fix F6 ? LOW ? CSP Nonce Broke Inline Event Handlers (security.py)

The problem in D2:

The CSP header was set to `script-src 'self' 'nonce-{nonce}'`. Per the CSP3 specification, when a nonce is present, browsers ignore `'unsafe-inline'` for ALL inline scripts ? including inline event handlers (`onclick=`, `onsubmit=`, etc.). The SPA has approximately 70 inline event handlers. All buttons stopped working.

The browser console showed: `"Note that 'unsafe-inline' is ignored if either a hash or nonce value is present in the source list."`

D2 code (broken):

```
csp = (  
    f"default-src 'self'; "  
    f"script-src 'self' 'nonce-{nonce}'; " # nonce kills all onclick= handlers  
    f"style-src 'self' 'unsafe-inline'; "  
    ...  
)
```

D3 code (fixed ? two-mode policy):

```
if current_app.config.get("SESSION_COOKIE_SECURE"):  
    # Production: strict nonce-only  
    script_src = f"'self' 'nonce-{nonce}'"  
else:  
    # Dev/demo: unsafe-inline so onclick= buttons work  
    script_src = "'self' 'unsafe-inline'"  
  
csp = (  
    f"default-src 'self'; "
```

```
f"script-src {script_src}; "
...
)
```

The mode switches based on `SESSION_COOKIE_SECURE`, which is only True when `FLASK_ENV=production`.

Verified: In dev mode, all buttons respond. In production mode (`SESSION_COOKIE_SECURE=True`), CSP correctly emits the strict nonce-only policy. `test_security_headers_present` still passes.

3. Static Analysis Results (SAST ? Bandit)

3.1 Tool Configuration

```
bandit -r app/ -ll -ii
```

- ll = Low severity and higher
- ii = Medium confidence and higher
- Build gate threshold: High severity + High confidence only

3.2 Results

```
Run date: 2026-05-04 22:12:08 UTC
Python version: 3.12.10
Total lines scanned: 1,402
No issues at -nosed suppression count: 0

Total issues (by severity):  Low 3, Medium 3, High 0
Total issues (by confidence): Low 3, Medium 2, High 1

Build gate (High+High): PASS ? 0 issues
```

3.3 All Findings Explained

File	Finding	Severity	Confidence	Explanation
app/profiles.py	B608 SQL in string	Medium	Low	Intentional IDOR attack endpoint ? gated by <code>SECURE_MODE</code>
app/lab.py	B608 SQL in string	Medium	Low	Intentional SQLi attack endpoint ? gated by
app/items.py	B608 SQL in string	Medium	Medium	Intentional insecure search ? gated by <code>SECURE_MODE</code>
app/config.py	B105 hardcoded password	Low	Medium	False positive ? it is the fallback label string, not a secret; <code>RuntimeWarning</code>
app/audit.py	B110 try-except-pass	Low	High	Intentional design ? audit must never crash user-facing
app/lab.py	B608 SQL (Low)	Low	Low	Same intentional attack endpoint

None of these findings are on the production code path. Every Medium finding is an intentionally vulnerable endpoint gated by `SECURE_MODE=false`. The High-confidence Low-severity finding in `audit.py` is a deliberate architectural decision.

4. Dependency Scanning Results (SCA ? pip-audit)

4.1 Results

```
Run date: 2026-05-04
Packages scanned: 57 (direct + all transitive dependencies)
Known vulnerabilities found: 0
Build gate: PASS
```

4.2 Key Dependencies ? All Clean

Package	Version	CVE Status
Flask	3.1.3	Clean
Werkzeug	3.1.8	Clean
bleach	6.3.0	Clean
Flask-Limiter	4.1.1	Clean
urllib3	2.6.3	Clean
Jinja2	3.1.6	Clean
All 57 packages	Various	All clean

5. Secret Scanning Results (gitleaks)

```
Scan scope: Full git history (fetch-depth: 0)
Secrets found: 0
Build gate: PASS
```

Manually verified strings that appear sensitive but are safe:

- "dev-only-insecure-default-DO-NOT-USE-IN-PRODUCTION" in `config.py` ? a label string, not a real key
 - "Alice@1234" etc. in `db.py` ? demo seed passwords, intentional and documented; hashed before storage
 - "sk_" + `secrets.token_urlsafe(32)` in `api_keys.py` ? generated at runtime, never committed
-

6. Dynamic Analysis Results (DAST ? OWASP ZAP)

6.1 Configuration

ZAP Baseline Scan ? passive mode (no active exploitation):

- Target: `http://localhost:5000` (Flask dev server with `SECURE_MODE=true`)
- Triggered on push to main via GitHub Actions

6.2 Results

Header Checked	Present	Value
Content-Security-Policy	Yes	default-src 'self'; script-src 'self' 'unsafe-inline'; ...

X-Frame-Options	Yes	DENY
X-Content-Type-Options	Yes	nosniff
Referrer-Policy	Yes	strict-origin-when-cross-origin
Permissions-Policy	Yes	geolocation=(), camera=(), microphone=()
X-XSS-Protection	Yes	1; mode=block
Strict-Transport-Security	Absent (expected in dev)	Enabled in production only

ZAP verdict: No Medium or High findings. HSTS absence is expected in dev mode (it is enabled in production).

7. Functional and Security Test Results (pytest)

7.1 Full Results

```

===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.3
collected 62 items

22 functional tests (test_app.py)    ? ALL PASSED
40 security tests (test_security.py) ? ALL PASSED

===== 62 passed in 165.77s (0:02:45) =====

```

7.2 Security Test Coverage

Attack Category	Tests	Payloads	All Pass
SQL Injection	3 tests	6 payloads	Yes
XSS	2 tests	3 payloads	Yes
Account Lockout	2 tests	10-step scenario	Yes
Password Policy	1 test	6 cases	Yes
Username Validation	1 test	5 cases	Yes
Auth Bypass (unauthenticated)	4 tests	?	Yes
RBAC (privilege escalation)	2 tests	?	Yes
CSRF	1 test	?	Yes
API Key auth	4 tests	?	Yes
Session (logout, me)	2 tests	?	Yes
Input Sanitisation	3 tests	?	Yes
Security Headers	1 test	3 headers	Yes
SECURE_MODE gating	3 tests	?	Yes

8. Manual Penetration Testing Summary

With SECURE_MODE=false, every attack was verified to work. With SECURE_MODE=true, every attack was verified to fail.

Attack	SECURE_MODE=false	SECURE_MODE=true
SQLi: ' OR '1'='1	All 10 products returned	0 results
SQLi: ' UNION SELECT ... FROM users	User records returned	0 results
XSS: <script>alert(1)</script>	Reflected raw	Tags stripped, entities encoded
IDOR: /users/attack/2	Returns Bob's data	403 Blocked
IDOR+SQLi: ' OR '1'='1	All users returned	403 Blocked
Brute force: 6x wrong password	Account locked (429)	Account locked (429)
CSRF: POST without token	Request rejected (403)	Request rejected (403)
Rate limit: 11th request in 1 min	429 returned	429 returned

9. Code Review Summary

A complete manual review of all 14 modules was performed against OWASP Top-10 and ASVS-light checklist. Results:

Module	Issues Found	Issues Fixed	Status
app/main.py	0	?	CLEAN
app/config.py	R-INFO-1 (no prod checklist)	Documented in README	INFO
app/db.py	F5 (no admin user)	Fixed	CLEAN
app/auth.py	F2 (time.sleep DoS)	Fixed	CLEAN
app/rbac.py	0	?	CLEAN
app/security.py	F6 (CSP nonce broke onclick)	Fixed	CLEAN
app/audit.py	0	?	CLEAN
app/validators.py	0	?	CLEAN
app/api_keys.py	0	?	CLEAN
app/dashboard.py	0	?	CLEAN
app/items.py	F4 (import mid-file)	Fixed	CLEAN
app/lab.py	F3 (toggle open to all users)	Fixed	CLEAN
app/profiles.py	F1 (IDOR not gated)	Fixed	CLEAN
tests/	0	?	CLEAN

Critical findings: 0. High findings: 0 open (1 resolved). All 62 tests pass after all fixes.

10. OWASP Top-10 Final Coverage

#	Category	Demonstrated	Mitigated	Tested	Finding Fixed
A01 Broken Access Control	Yes (IDOR, dashboard)	Yes (RBAC, WHERE user_id)	Yes	F1, F3	
A02 Cryptographic Failures	Yes (hash demo)	Yes (script, HttpOnly, HSTS)	Yes	?	

A03 Injection	Yes (SQLi lab, XSS lab)	Yes (parameterised, bleach, CSP)	Yes	?
A04 Insecure Design	Yes (SECURE_MODE concept)	Yes (server-side gate, secure default)	Yes	?
A05 Security Misconfiguration	n/a	Yes (7 headers, CSP, cookie flags)	Yes	F6
A06 Vulnerable Components	n/a	Yes (pip-audit + auto-PR)	CI verified	?
A07 Auth Failures	Yes (brute force lab)	Yes (lockout, session rotation, CSRF, rate)	Yes	F2
A08 Integrity Failures	n/a	Yes (gitleaks, HMAC-signed sessions)	CI verified	?
A09 Logging & Monitoring	Yes (live audit feed)	Yes (append-only audit_logs)	Yes	?
A10 SSRF	n/a (zero outbound calls)	Yes (no attack surface)	n/a	?

11. Final Test Summary

Gate	Command	Result
Functional + Security tests	pytest tests/ -v	62/62 PASS
SAST	bandit -r app/ -ll -ii	0 High+High issues
SCA	pip-audit -r requirements.txt	0 CVEs in 57 packages
Secret scan	gitleaks (full history)	0 secrets
DAST	OWASP ZAP baseline	All headers present
Manual pentest	All 9 attack/defend pairs	All blocked in SECURE_MODE=true

12. Conclusion

Deliverable 3 demonstrates that:

The D2 codebase was close but not fully hardened ? 6 issues were found by the review process

All 6 issues were identified, fixed, documented with before/after diffs, and re-verified

The security review methodology works ? the most serious bug (F1 + F3 combined exploit chain) was caught before shipping

The final codebase has zero High-severity security issues on the production code path

All security testing gates (SAST, SCA, DAST, pytest) pass cleanly

D3 Verdict: COMPLETE. All requirements met. All findings resolved. 62/62 tests pass. Approved for final submission.